

Parallel Architectures

Pacheco, Chapter 2 except 2.3.3

Moreno Marzolla
Dip. di Informatica—Scienza e Ingegneria (DISI)
Università di Bologna

Copyright © 2013—2025

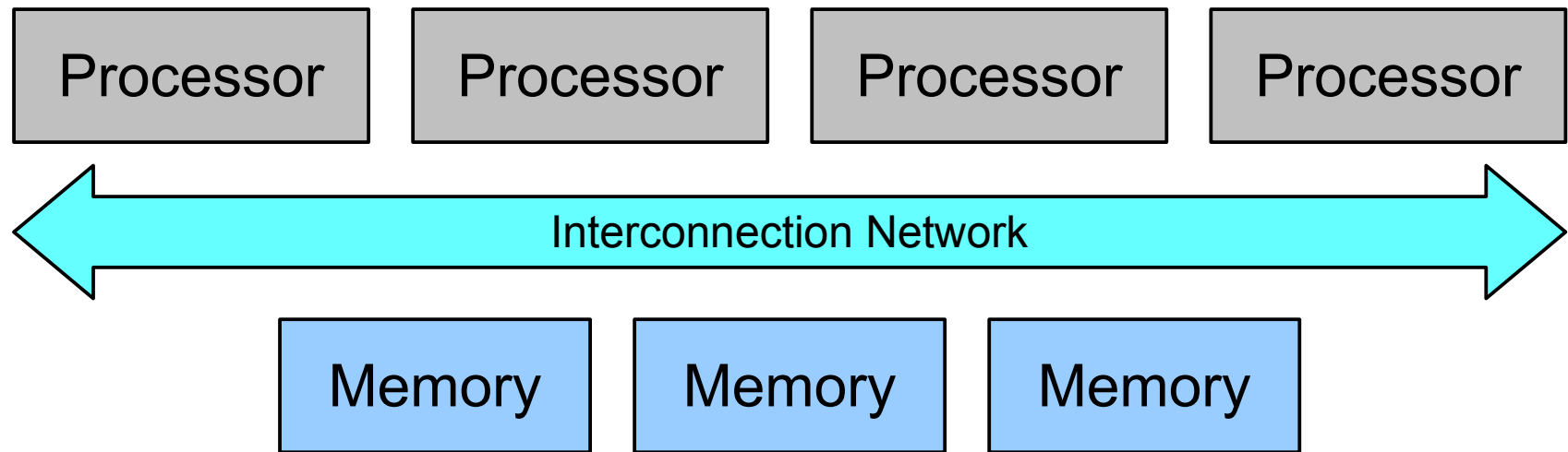
Moreno Marzolla, Università di Bologna, Italy

<https://www.moreno.marzolla.name/teaching/HPC/>



Except where stated otherwise, this work is licensed under the Creative Commons Attribution-ShareAlike 4.0 International License (CC BY-SA 4.0). To view a copy of this license, visit <https://creativecommons.org/licenses/by-sa/4.0/> or send a letter to Creative Commons, 543 Howard Street, 5th Floor, San Francisco, California, 94105, USA.

An Abstract Parallel Architecture



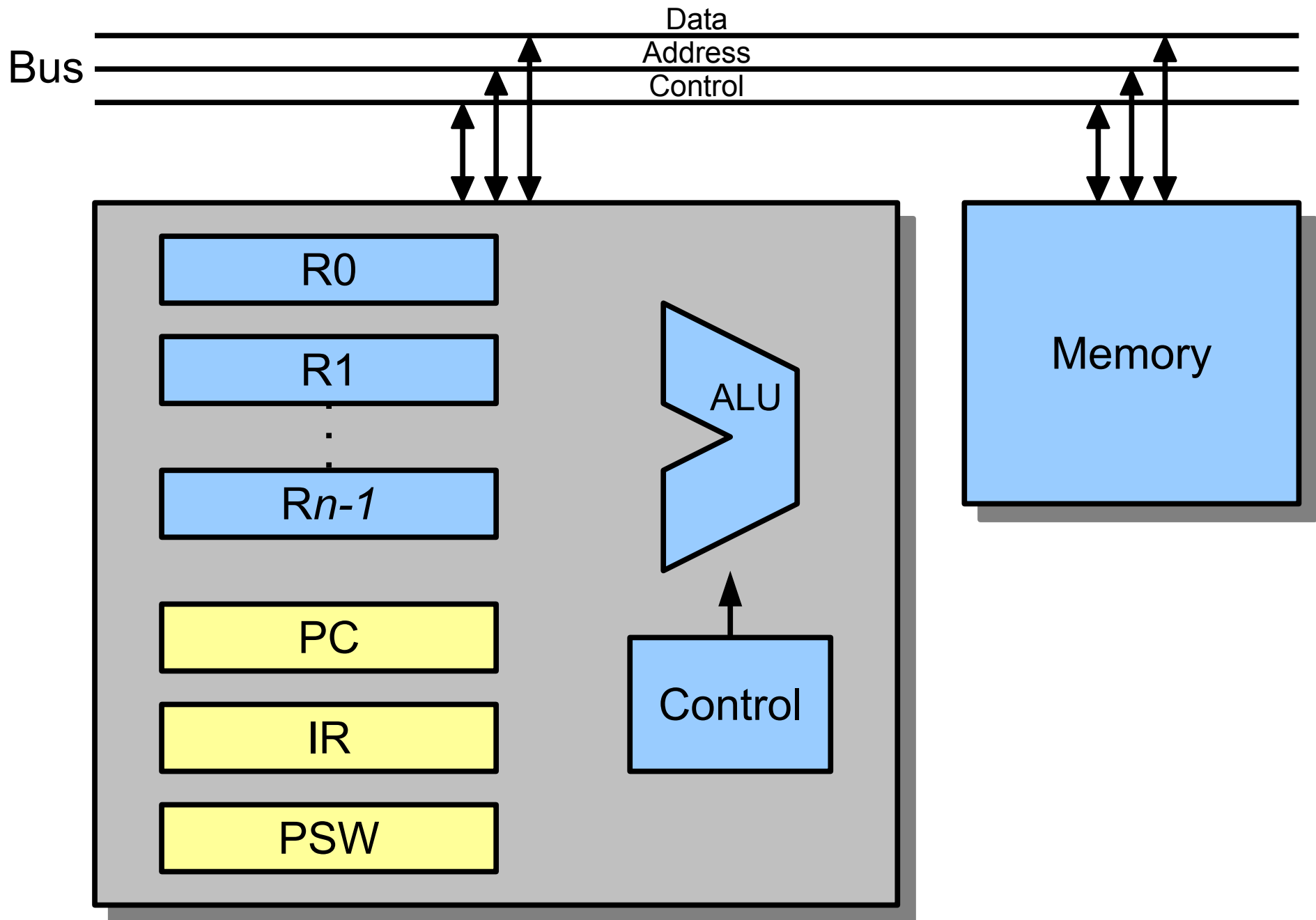
- How is parallelism handled?
- What is the exact physical location of the memories?
- What is the topology of the interconnect network?

Why are parallel architectures important?

- There is no "typical" parallel computer: different vendors use different architectures
- There is currently no “universal” programming paradigm that fits all architectures
 - Parallel programs must be tailored to the underlying parallel architecture
 - The architecture of a parallel computer limits the choice of the programming paradigm that can be used

Von Neumann architecture and its extensions

Von Neumann architecture



CPU times compared to the real world

	CPU	Real world
1 CPU cycle	0.3 ns	1 s
Level 1 cache access	0.9 ns	3 s
Level 2 cache access	2.8 ns	9 s
Level 3 cache access	12.9 ns	43 s
Main memory access	120 ns	6 min
Solid-state disk I/O	50-150 μ s	2-6 days
Rotational disk I/O	1-10 ms	1-12 months
Internet: SF to NYC	40 ms	4 years
Internet: SF to UK	81 ms	8 years
Internet: SF to Australia	183 ms	19 years
Physical system reboot	1 min	6 millennia

How to address the bottlenecks of Von Neumann architecture

- Reduce the memory access latency
 - Rely on CPU registers whenever possible
 - Use caches
- Hide the memory access latency
 - *Multithreading* and *context-switch* during memory accesses
- Execute multiple instructions at the same time
 - *Pipelining*
 - *Multiple issue*
 - *Branch prediction*
 - *Speculative execution*
 - *SIMD extensions*

Caching

Processor/Memory wall

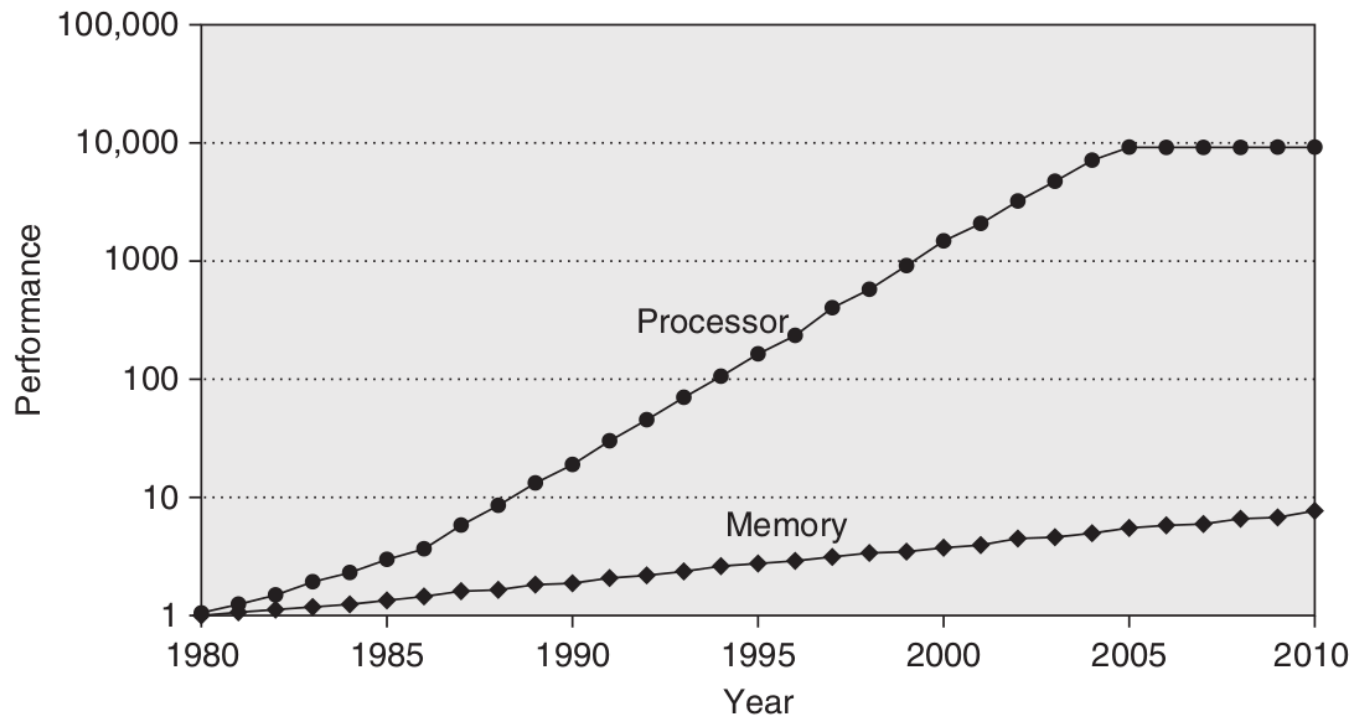
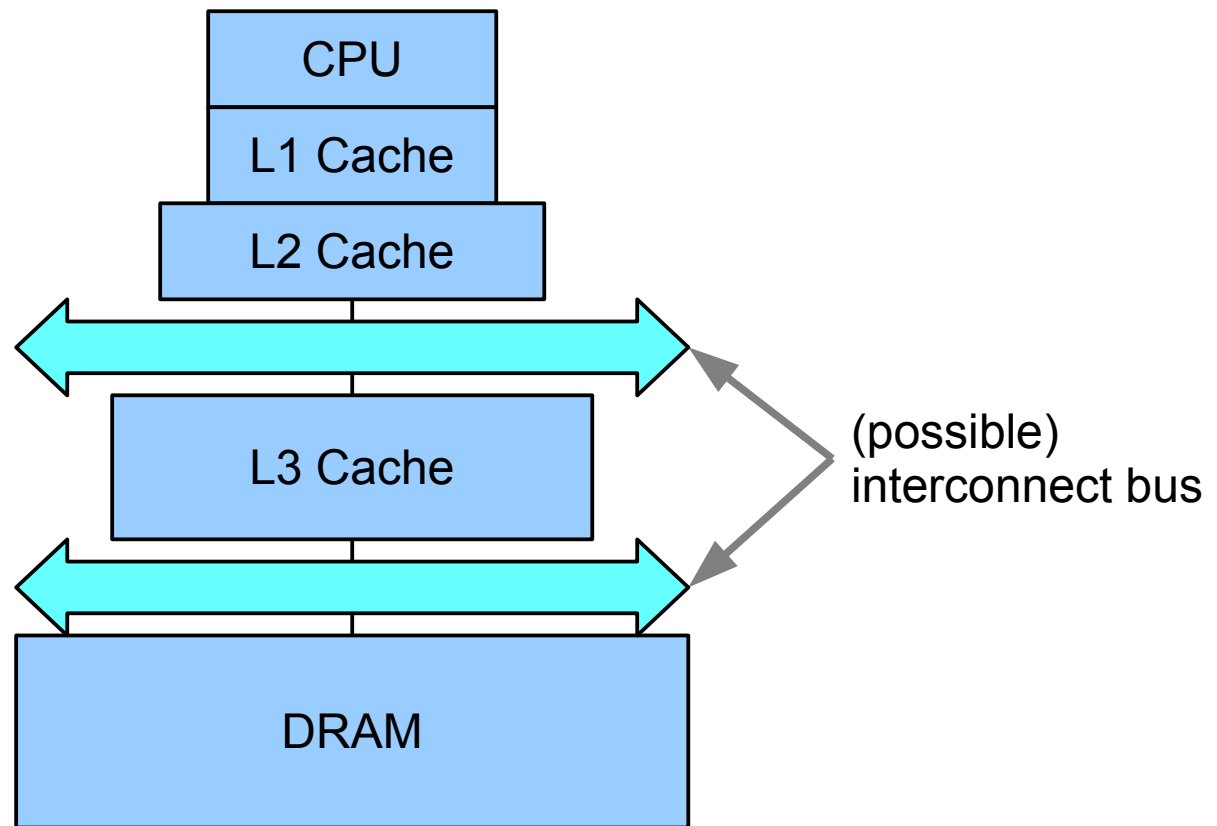


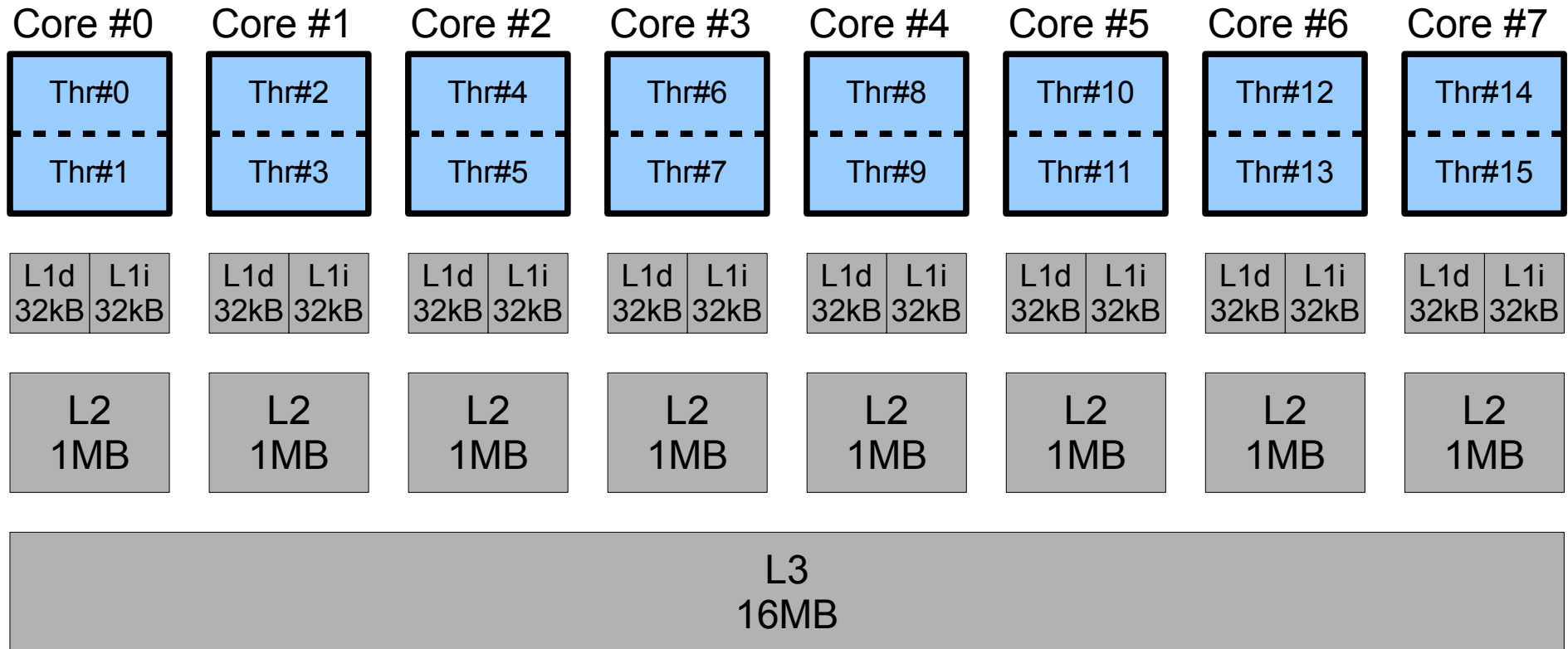
Figure 2.2 Starting with 1980 performance as a baseline, the gap in performance, measured as the difference in the time between processor memory requests (for a single processor or core) and the latency of a DRAM access, is plotted over time. Note that the vertical axis must be on a logarithmic scale to record the size of the processor–DRAM performance gap. The memory baseline is 64 KB DRAM in 1980, with a 1.07 per year performance improvement in latency (see [Figure 2.13](#) on page 99). The processor line assumes a 1.25 improvement per year until 1986, a 1.52 improvement until 2000, a 1.20 improvement between 2000 and 2005, and no change in processor performance (on a per-core basis) between 2005 and 2010; see [Figure 1.1](#) in [Chapter 1](#).

Cache hierarchy

- Large memories are slow; fast memories are small



Example of cache hierarchy

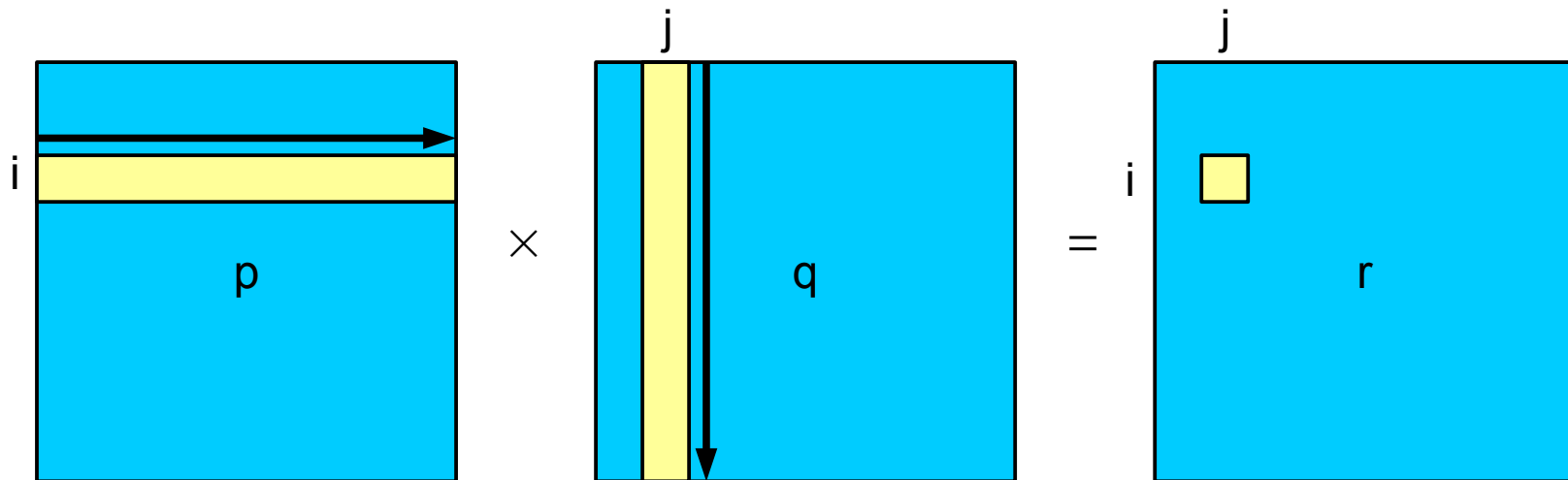


Spatial and temporal locality

- Cache memory works well when applications exhibit **spatial** and/or **temporal** locality
- **Spatial locality**
 - Accessing adjacent memory locations is OK
- **Temporal locality**
 - Repeatedly accessing the same memory location(s) is OK

Example: matrix-matrix product

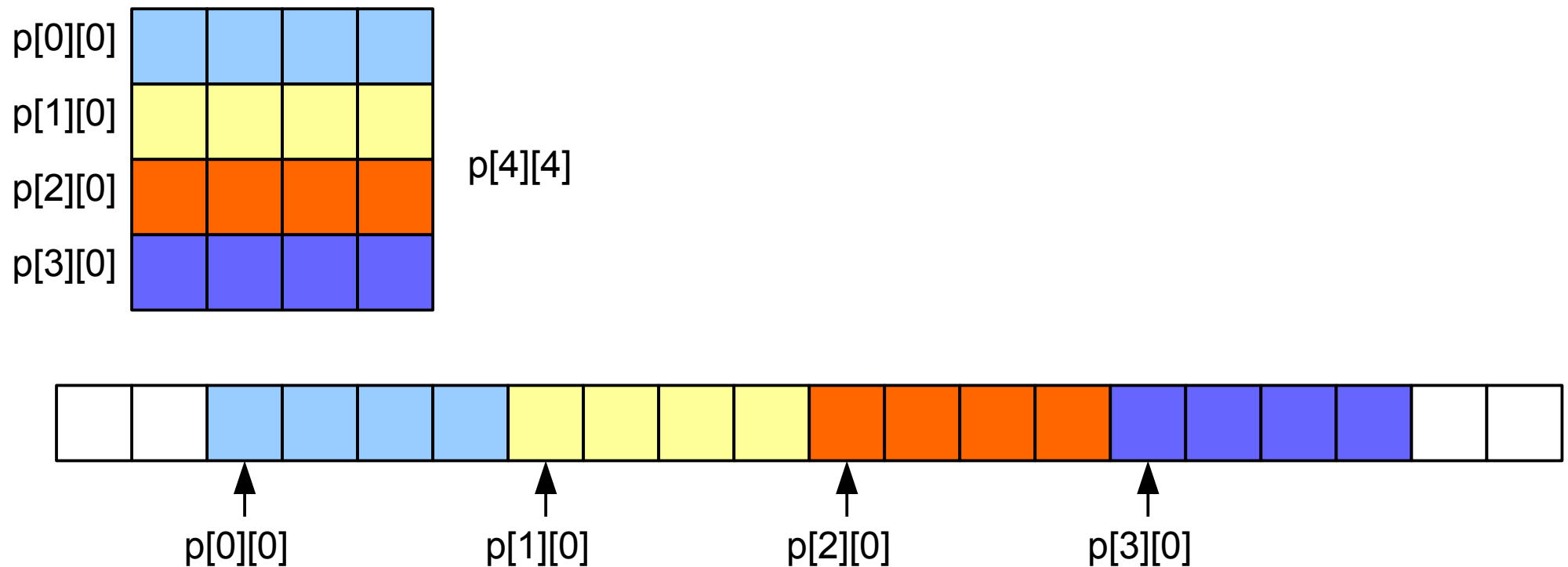
- Given two square matrices p , q , compute $r = p \times q$



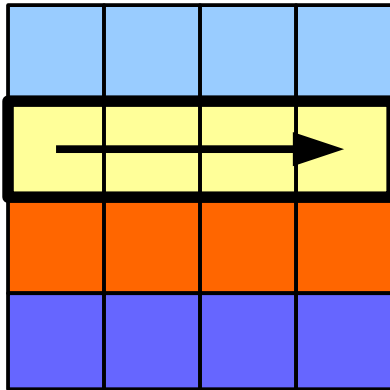
```
void matmul( double *p, double* q, double *r, int n)
{
    int i, j, k;
    for (i=0; i<n; i++) {
        for (j=0; j<n; j++) {
            double s = 0.0;
            for (k=0; k<n; k++) { s += p[i*n + k] * q[k*n + j]; }
            r[i*n + j] = s;
        }
    }
}
```

Matrix representation

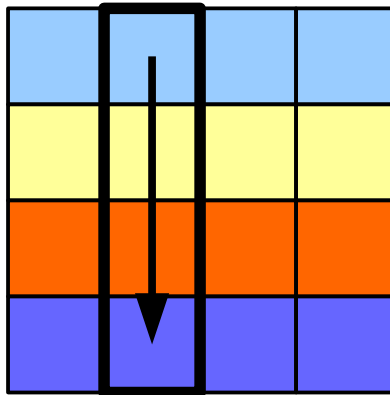
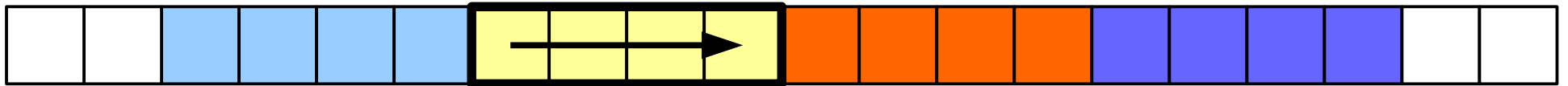
- Matrices in C are stored in *row-major order*
 - Elements of each row are contiguous in memory
 - Adjacent rows are contiguous in memory



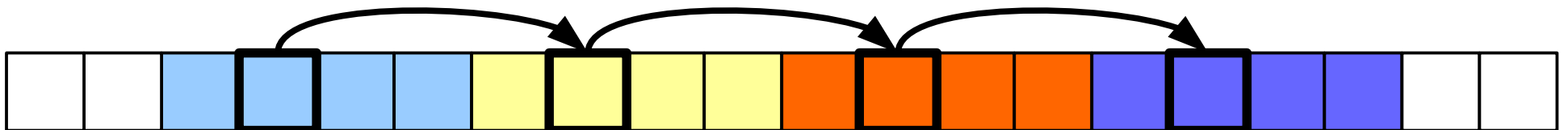
Row-wise vs column-wise access



Row-wise access is OK: the data is contiguous in memory, so the cache helps (spatial locality)

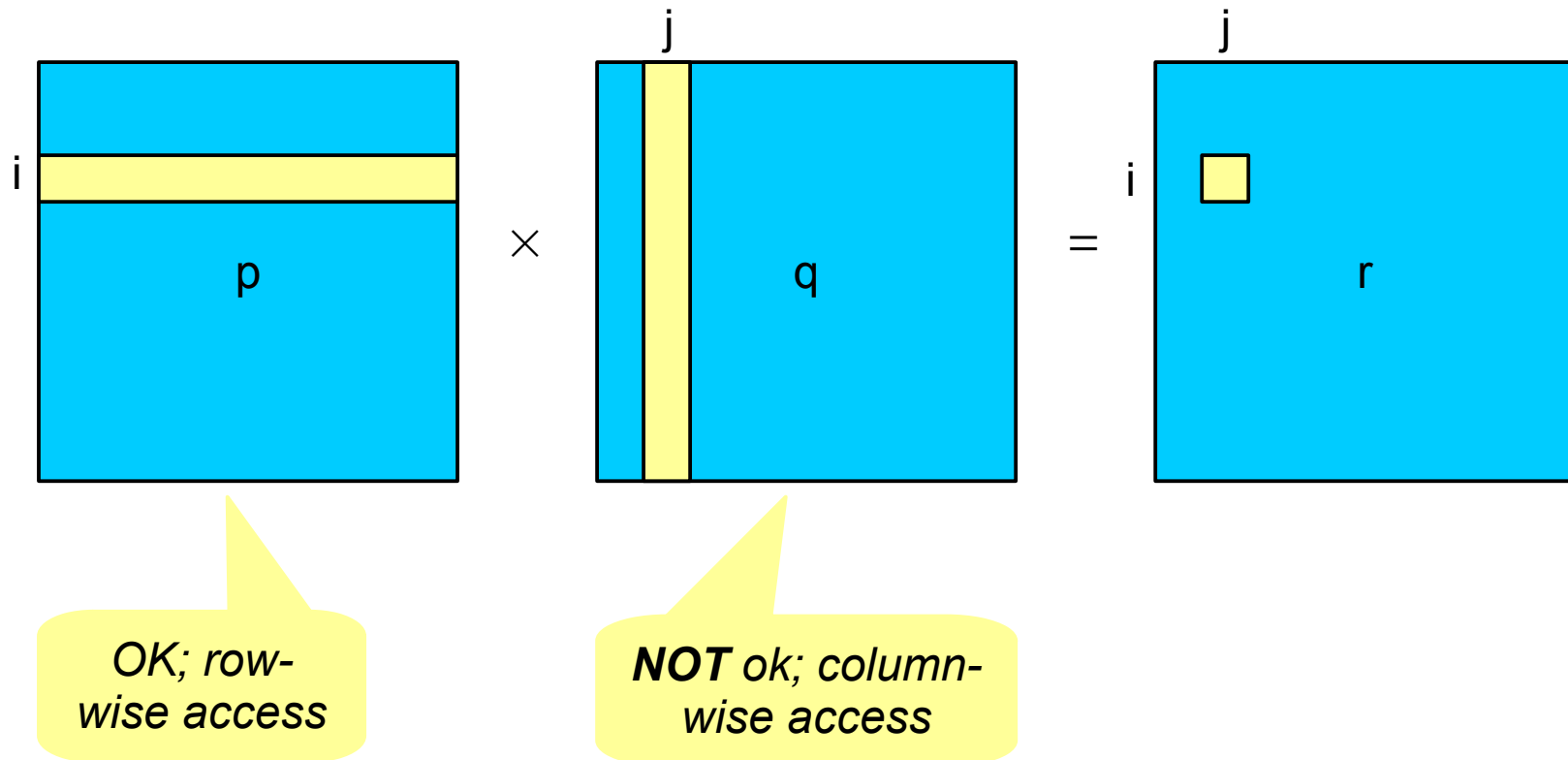


Column-wise access is **NOT OK**: the accessed elements are not contiguous in memory (*strided* access) so the cache does NOT help



Matrix-matrix product

- Given two squared matrices p , q , compute $r = p \times q$



Optimizing the memory access pattern

p00	p01	p02	p03
p10	p11	p12	p13
p20	p21	p22	p23
p30	p31	p32	p33

 $\times$

q00	q01	q02	q03
q10	q11	q12	q13
q20	q21	q22	q23
q30	q31	q32	q33

 $=$

r00	r01	r02	r03
r10	r11	r12	r13
r20	r21	r22	r23
r30	r31	r32	r33

p q r

 Transpose q

p00	p01	p02	p03
p10	p11	p12	p13
p20	p21	p22	p23
p30	p31	p32	p33

 $\times$

q00	q10	q20	q30
q01	q11	q21	q31
q02	q12	q22	q32
q03	q13	q23	q33

 $=$

r00	r01	r02	r03
r10	r11	r12	r13
r20	r21	r22	r23
r30	r31	r32	r33

p q^T r

But...

- Transposing q requires time. Is it worth it?
- `make serial` will produce two executables, `matmul-plain` and `matmul-transpose`
- To measure the cache performance we use `perf`
 - On Debian/Ubuntu:
`sudo apt-get install linux-tools-common linux-tools-generic`
 - Which counters can be measured is system-dependent 😞
 - The meaning of counters is system-dependent 😞😞😞

Measuring cache stats

```
perf stat -B -e \  
    task-clock,cycles,cache-references,\  
    cache-misses,L1-dcache-loads,L1-dcache-load-misses,\  
    L1-dcache-stores,L1-dcache-store-misses \  
    ./exec_name
```

```
$ ./cache-stats.sh ./matmul-plain
```

```
Starting plain matrix-matrix multiply (n=1000)... done
```

```
r[0][0] = 243.361596
```

```
elapsed time = 3.894047 s
```

*Low level cache
references and misses*

```
Performance counter stats for './matmul-plain':
```

3909,022101	task-clock (msec)	#	0,997	CPUs utilized
13.544.312.192	cycles	#	3,465	GHz
126.008.898	cache-references	#	32,235	M/sec
33.267.819	cache-misses	#	26,401	% of all cache refs
14.061.431.913	L1-dcache-loads	#	3597,174	M/sec
1.129.759.631	L1-dcache-load-misses	#	8,03%	of all L1-dcache hits
0	L1-dcache-stores	#	0,000	K/sec
1.776.808	L1-dcache-store-misses	#	0,455	M/sec

```
3,919924574 seconds time elapsed
```

*L1 data cache references
and misses*

Measuring cache stats

```
perf stat -B -e \  
    task-clock,cycles,cache-references,\  
    cache-misses,L1-dcache-loads,L1-dcache-load-misses,\  
    L1-dcache-stores,L1-dcache-store-misses \  
    ./exec_name
```

```
$ ./cache-stats.sh ./matmul-transpose
```

```
Starting transpose matrix-matrix multiply (n=1000)... done
```

```
r[0][0] = 243.361596
```

```
elapsed time = 2.656615 s
```

*Low level cache
references and misses*

```
Performance counter stats for './matmul-transpose'
```

2678,964056	task-clock (msec)	#	0,999	CPUs utilized
9.271.087.662	cycles	#	3,461	GHz
3.305.170	cache-references	#	1,234	M/sec
1.292.062	cache-misses	#	39,092	% of all cache refs
14.073.347.851	L1-dcache-loads	#	5253,280	M/sec
127.489.346	L1-dcache-load-misses	#	0,91%	of all L1-dcache hits
0	L1-dcache-stores	#	0,000	K/sec
1.960.718	L1-dcache-store-misses	#	0,732	M/sec

```
2,682123014 seconds time elapsed
```

*L1 data cache references
and misses*

Hardware Multithreading

Hardware multithreading

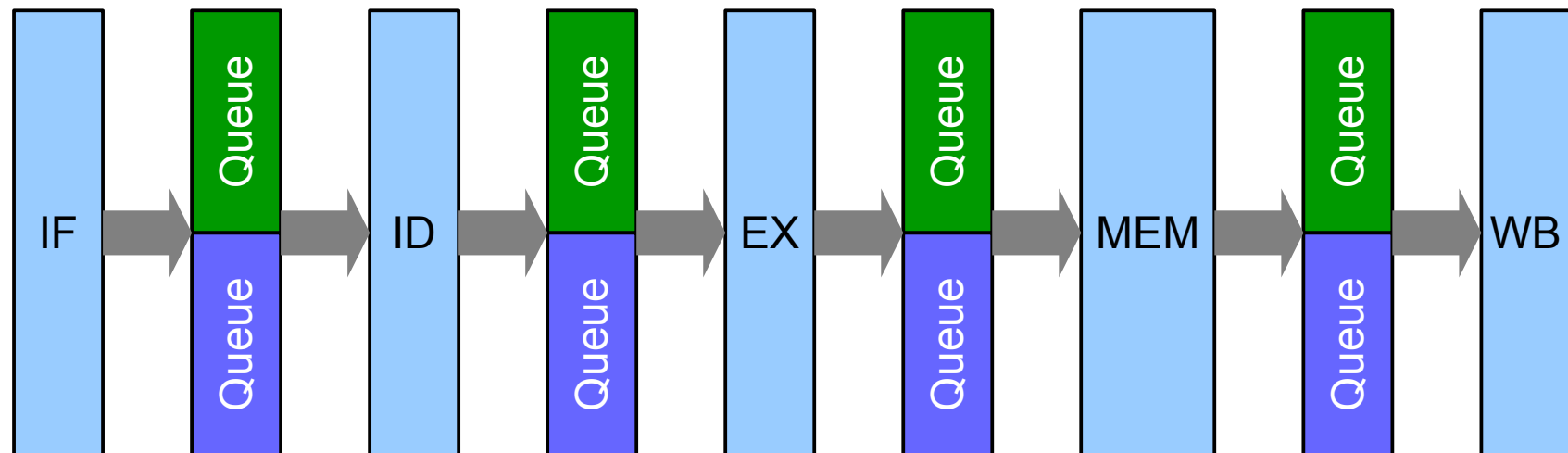
- Allows the CPU to switch to another task when the current task is stalled
- **Fine-grained multithreading**
 - *A context switch* has essentially zero cost
 - The CPU can switch to another task even on stalls of short durations (e.g., waiting for memory operations)
 - Requires specific CPU support, e.g., Cray XMT
- **Coarse-grained multithreading**
 - *A context switch* has non-negligible cost, and is appropriate only for threads blocked on I/O operations or similar
 - The CPU is less efficient in the presence of stalls of short duration

Hardware multithreading

- *Simultaneous multithreading* (SMT) is an implementation of *fine-grained multithreading* where different threads can use multiple different units at the same time
- *HyperThreading* is Intel's implementation of SMT
 - Each physical processor core is seen by the Operating System as *two* "logical" processors
 - Each "logical" processor maintains a complete set of the architecture state:
 - general-purpose registers
 - control registers
 - advanced programmable interrupt controller (APIC) registers
 - some machine state registers
 - Intel claims that HT provides a 15—30% speedup with respect to an equivalent, non-HT processor

HyperThreading

- The pipeline stages are separated by two buffers (one for each executing thread)
- If one thread is stalled, the other one might go ahead and fill the pipeline slot, improving processor utilization



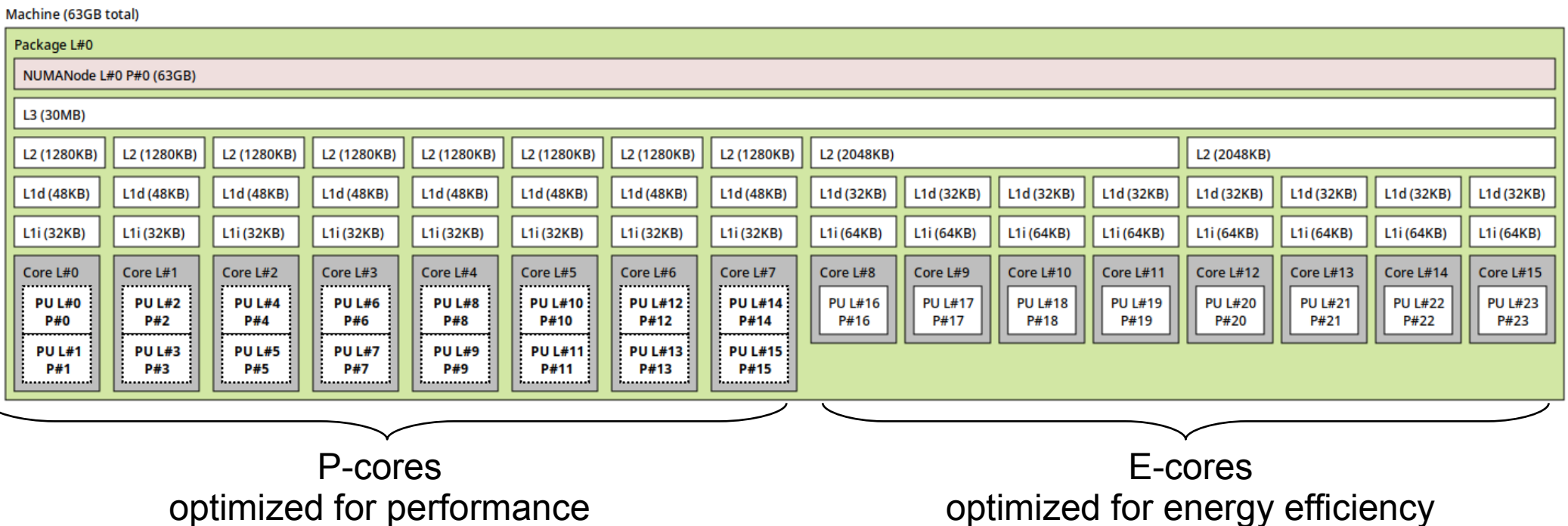
Hyper-Threading Technology Architecture and Microarchitecture

http://www.cs.virginia.edu/~mc2zk/cs451/vol6iss1_art01.pdf

Example

Hyperthreading + Asymmetric cores

- Output of `lscpu` ("Thread(s) per core") or `lstopo` (*hwloc* Ubuntu/Debian package)



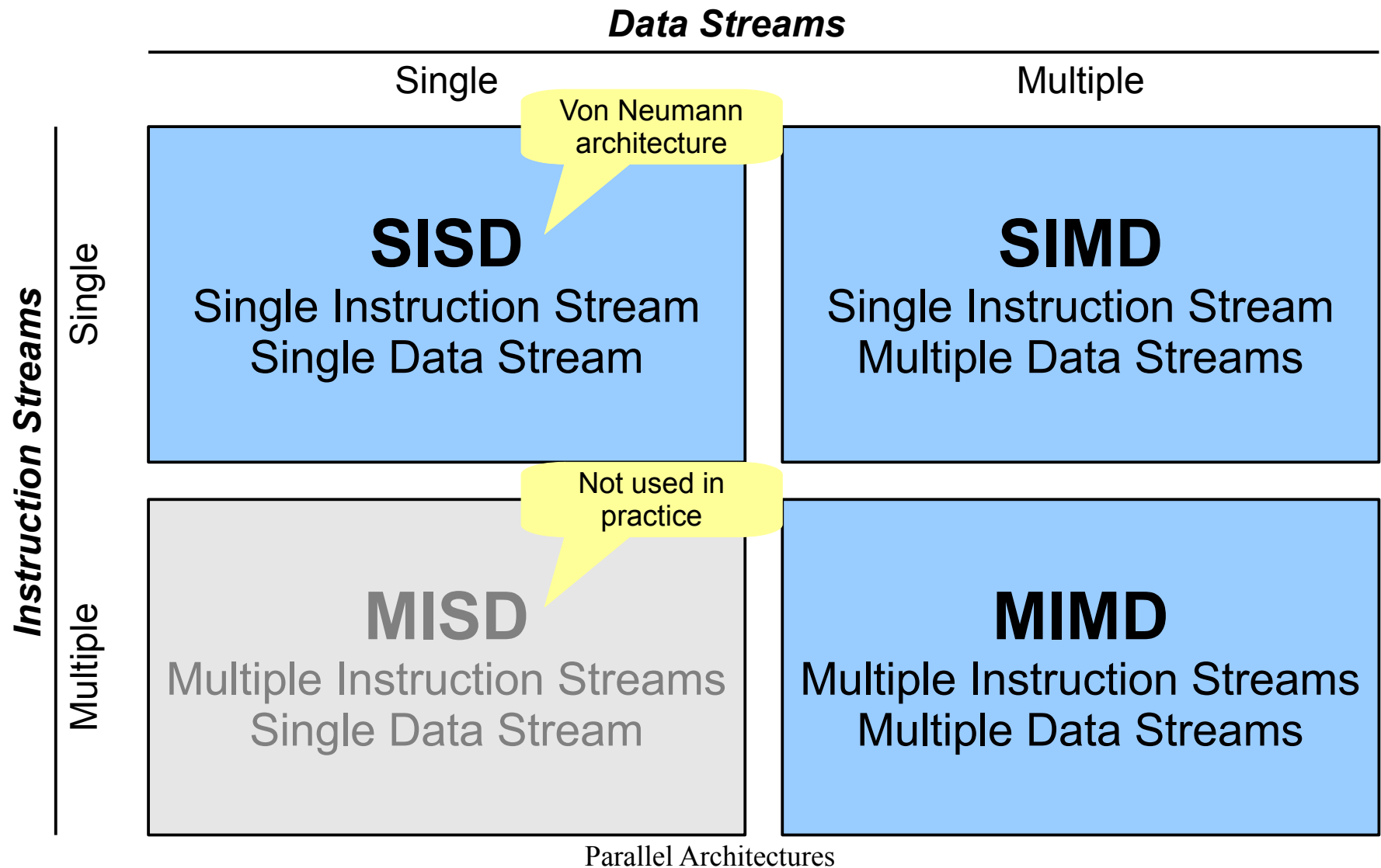
P-cores and E-cores

```
$ lscpu --all --extended
```

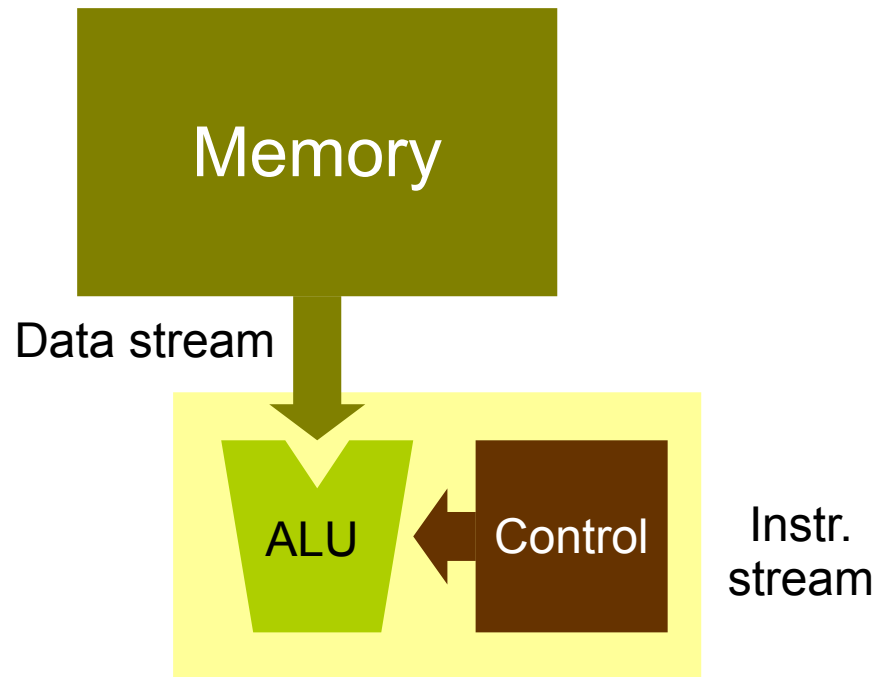
CPU	NODE	SOCKET	CORE	L1d:L1i:L2:L3	ONLINE	MAXMHZ	MINMHZ	MHZ
0	0	0	0	0:0:0:0	yes	5000,0000	800,0000	799,4240
1	0	0	0	0:0:0:0	yes	5000,0000	800,0000	800,0000
2	0	0	1	4:4:1:0	yes	5000,0000	800,0000	800,0000
3	0	0	1	4:4:1:0	yes	5000,0000	800,0000	800,0000
4	0	0	2	8:8:2:0	yes	5000,0000	800,0000	800,0000
5	0	0	2	8:8:2:0	yes	5000,0000	800,0000	799,9740
6	0	0	3	12:12:3:0	yes	5000,0000	800,0000	800,0000
7	0	0	3	12:12:3:0	yes	5000,0000	800,0000	799,8100
8	0	0	4	16:16:4:0	yes	5100,0000	800,0000	899,9900
9	0	0	4	16:16:4:0	yes	5100,0000	800,0000	899,0870
10	0	0	5	20:20:5:0	yes	5100,0000	800,0000	853,4400
11	0	0	5	20:20:5:0	yes	5100,0000	800,0000	881,8960
12	0	0	6	24:24:6:0	yes	5000,0000	800,0000	800,0000
13	0	0	6	24:24:6:0	yes	5000,0000	800,0000	800,0000
14	0	0	7	28:28:7:0	yes	5000,0000	800,0000	799,9210
15	0	0	7	28:28:7:0	yes	5000,0000	800,0000	800,0000
16	0	0	8	32:32:8:0	yes	3800,0000	800,0000	799,9110
17	0	0	9	33:33:8:0	yes	3800,0000	800,0000	800,0050
18	0	0	10	34:34:8:0	yes	3800,0000	800,0000	800,0090
19	0	0	11	35:35:8:0	yes	3800,0000	800,0000	799,9380
20	0	0	12	36:36:9:0	yes	3800,0000	800,0000	799,8960
21	0	0	13	37:37:9:0	yes	3800,0000	800,0000	801,0910
22	0	0	14	38:38:9:0	yes	3800,0000	800,0000	801,2520
23	0	0	15	39:39:9:0	yes	3800,0000	800,0000	800,0000

Parallel Architectures

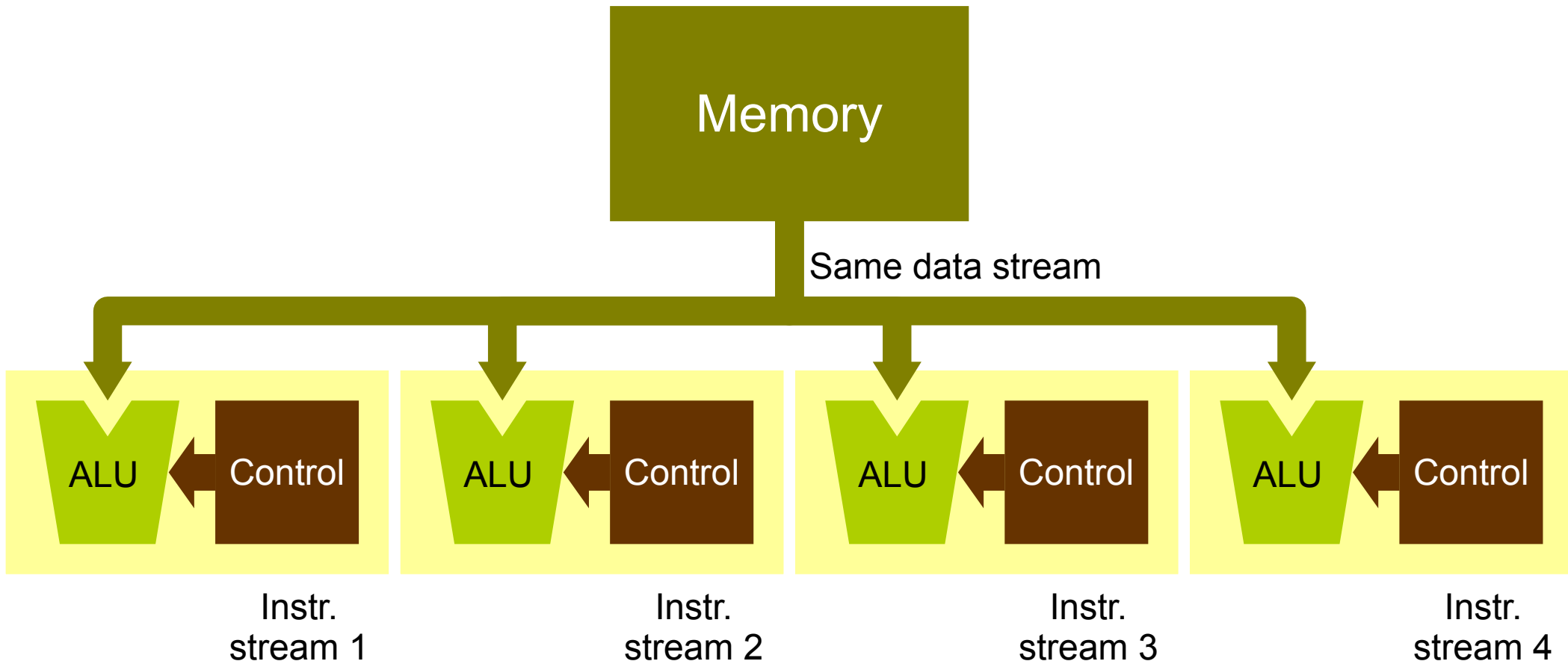
Flynn's Taxonomy



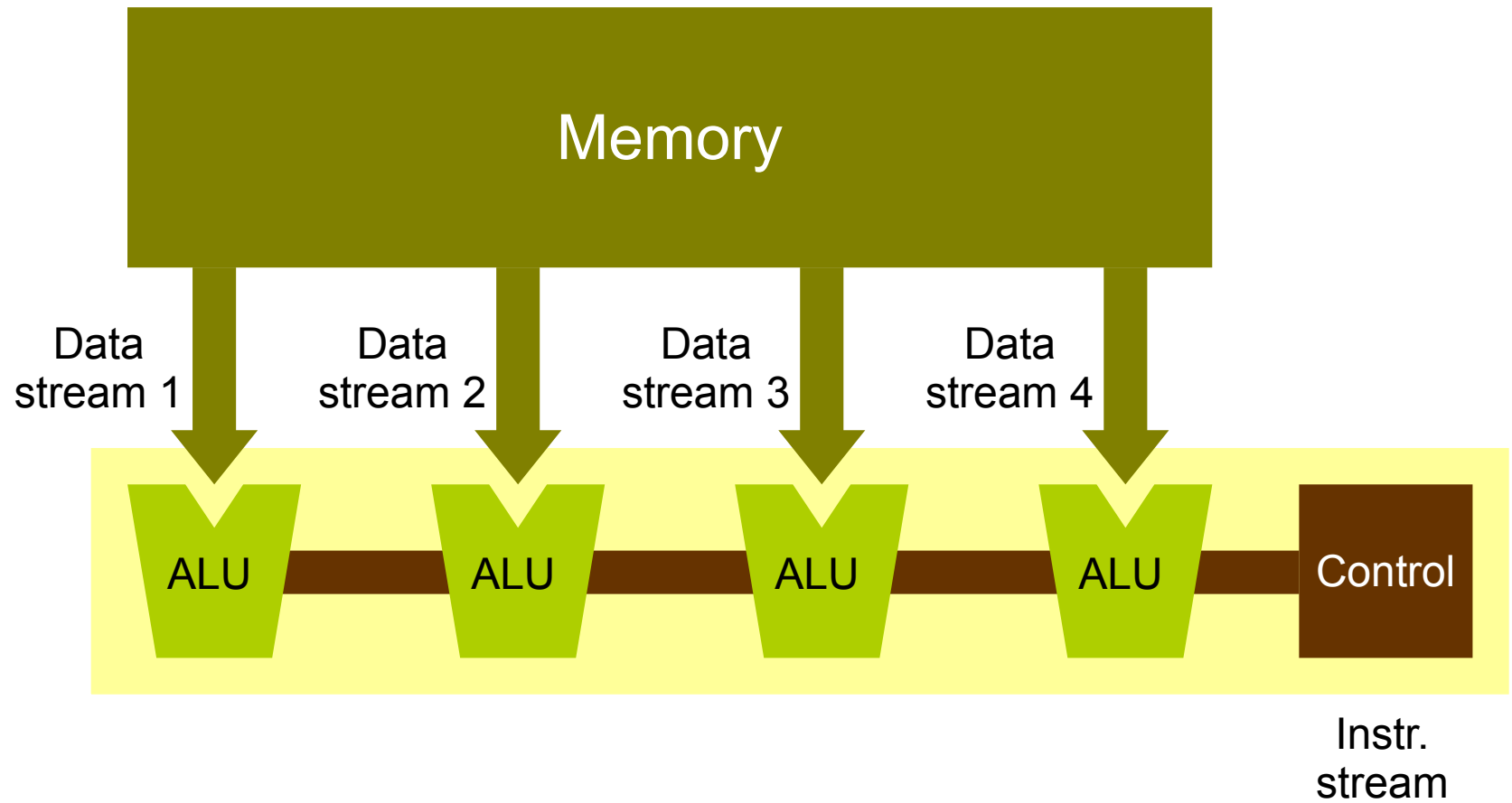
SISD



MISD

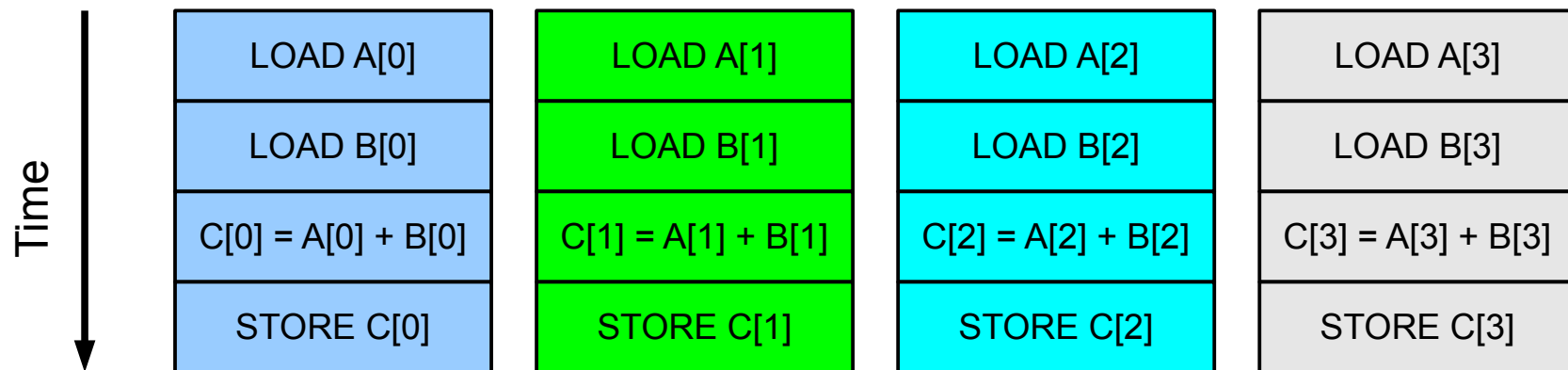


SIMD



SIMD

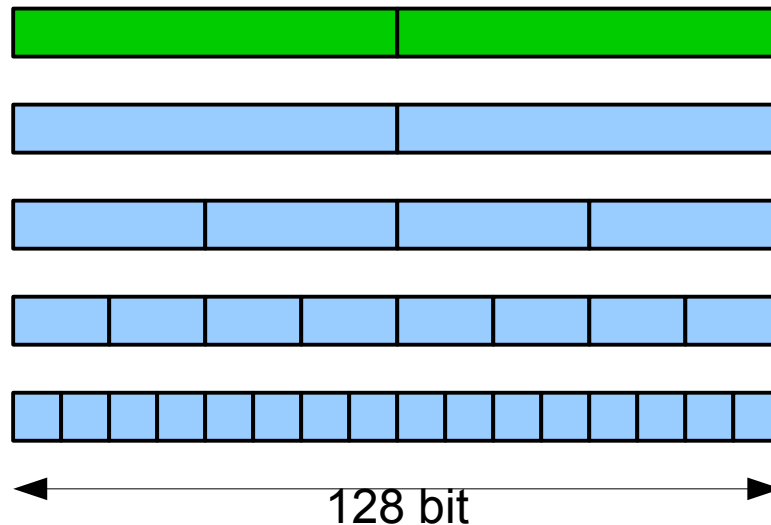
- SIMD instructions apply the **same** operation (e.g., sum, product...) to multiple elements (typically 4 or 8, depending on the width of SIMD registers and the data types of operands)
 - This means that there must be 4/8/... ALUs



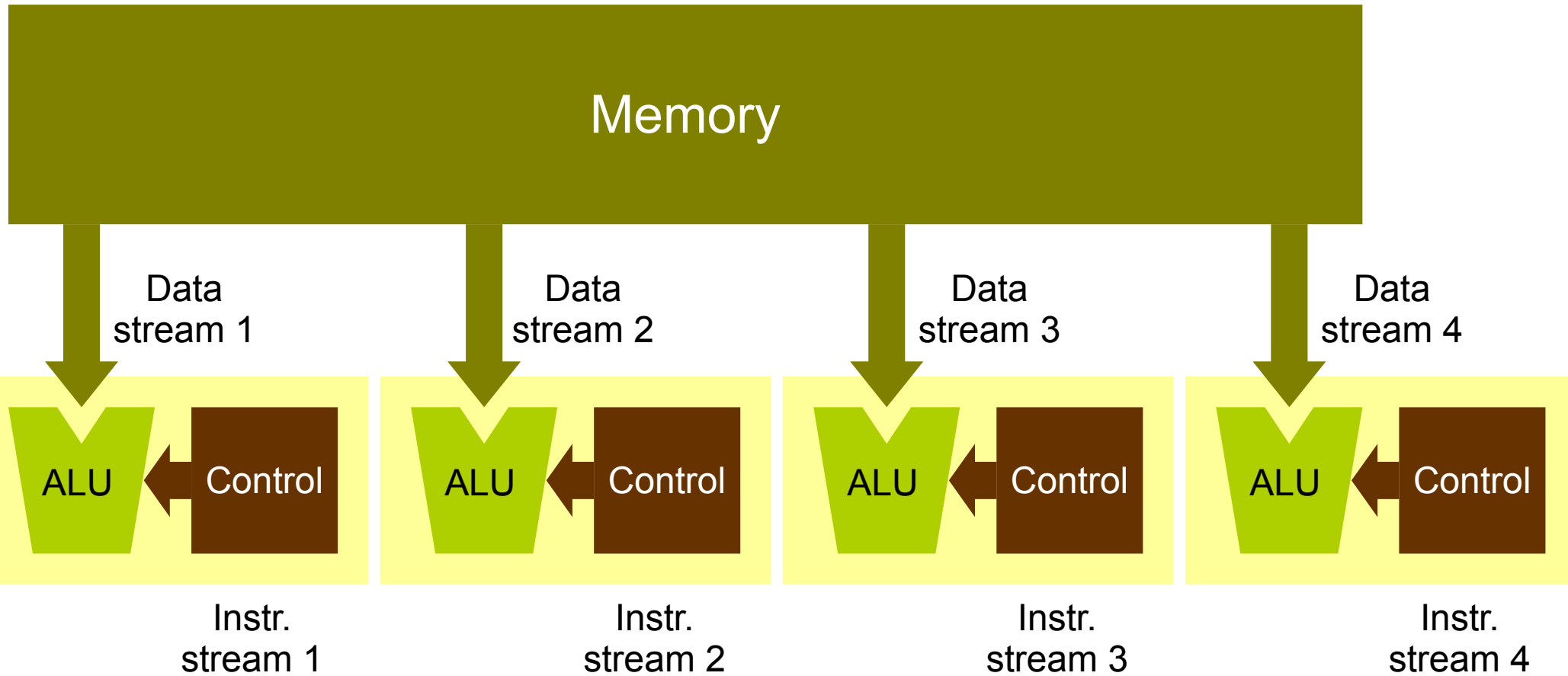
SSE

Streaming SIMD Extensions

- Extension to the x86 instruction set
- Provide new SIMD instructions operating on small arrays of integer or floating-point numbers
- Introduces 8 new 128-bit registers (XMM0—XMM7)
- SSE2 instructions can handle
 - 2 64-bit doubles, or
 - 2 64-bit integers, or
 - 4 32-bit integers, or
 - 8 16-bit integers, or
 - 16 8-bit chars

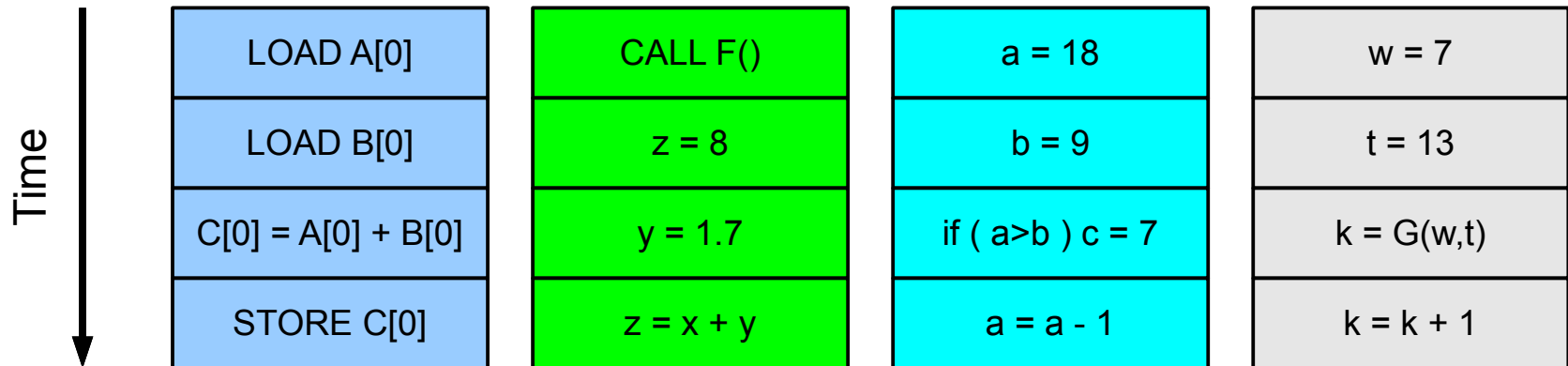


MIMD



MIMD

- In MIMD systems there are multiple execution units that can execute multiple sequences of instructions
 - Multiple Instruction Streams
- Each execution unit generally operates on its own input data
 - Multiple Data Streams



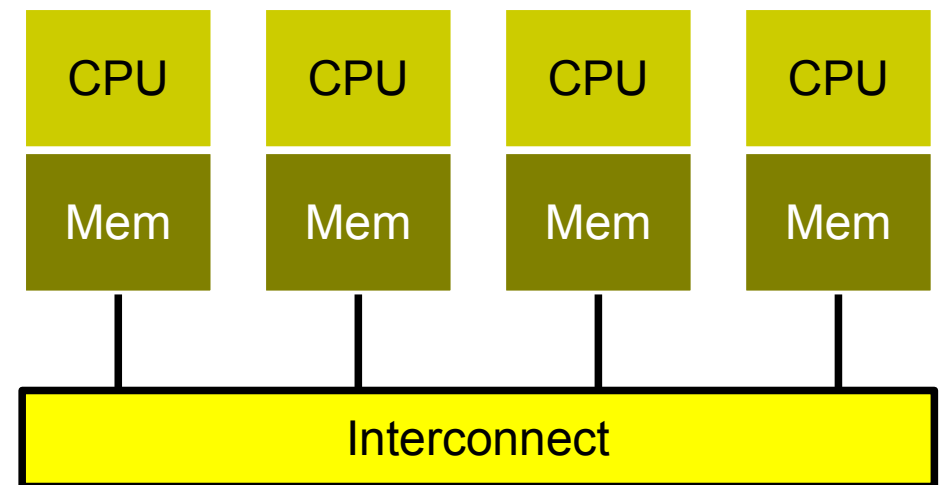
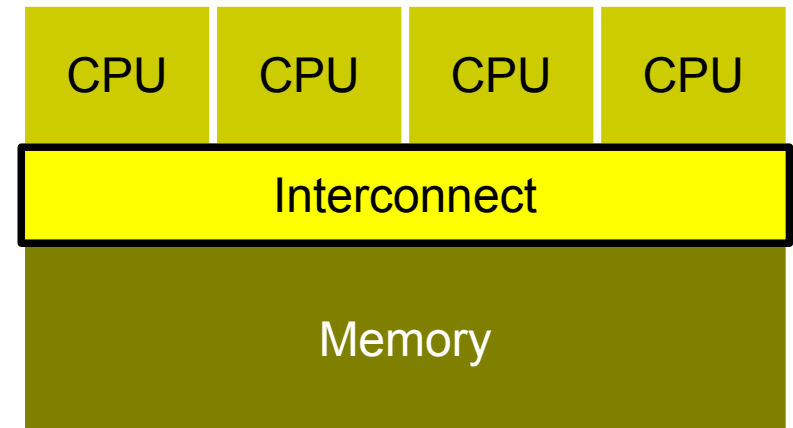
MIMD architectures

- **Shared Memory**

- A set of processors sharing a common memory space
- Each processor can access any memory location

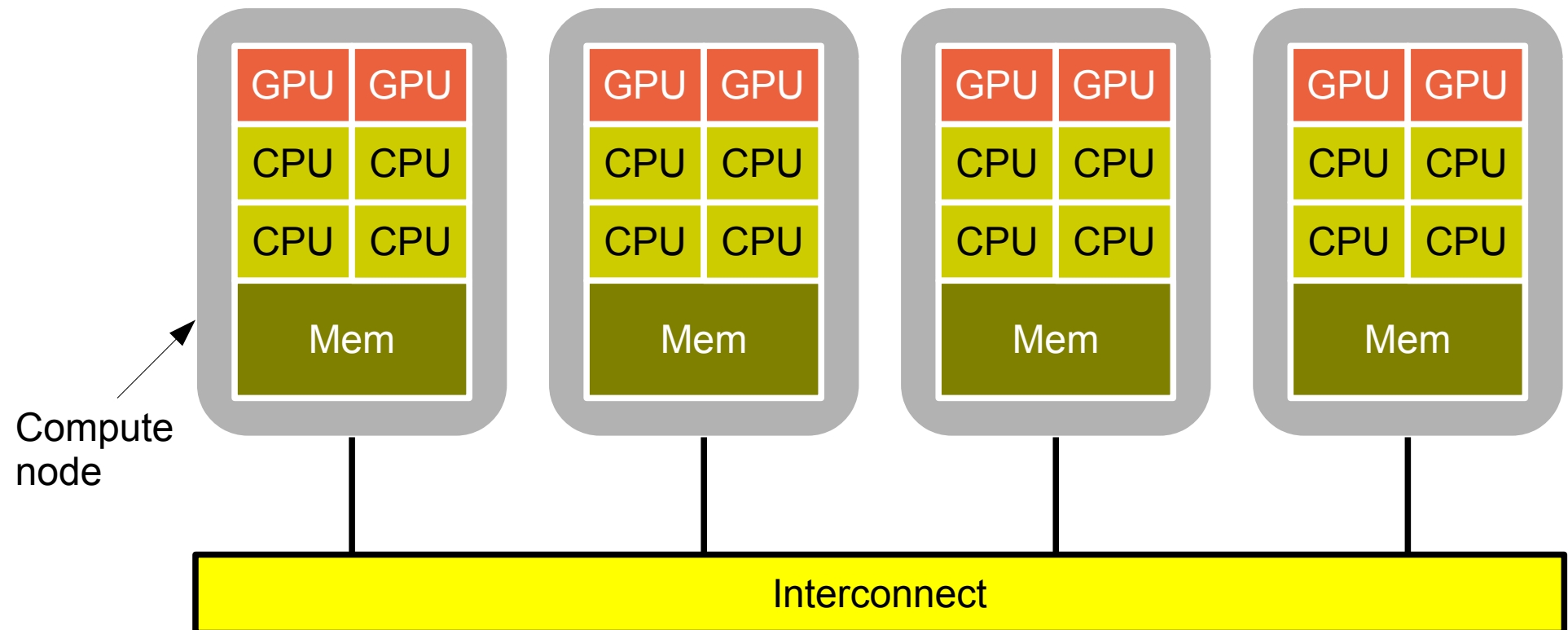
- **Distributed Memory**

- A set of compute nodes connected through an interconnection network
 - The most simple example: cluster of PCs connected via Ethernet
- Nodes can share data through explicit communications



Hybrid architectures

- Many HPC systems are based on hybrid architectures
 - Each compute node is a shared-memory multiprocessor
 - A large number of compute nodes is connected through an interconnect network



www.top500.org

(June 2025)



Rank	System	Cores	Rmax (PFlop/s)	Rpeak (PFlop/s)	Power (kW)
1	El Capitan - HPE Cray EX255a, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A, Slingshot-11, TOSS, HPE DOE/NNSA/LLNL United States	11,039,616	1,742.00	2,746.38	29,581



2	Frontier - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE Cray OS, HPE DOE/SC/Oak Ridge National Laboratory United States	9,066,176	1,353.00	2,055.72	24,607
---	--	-----------	----------	----------	--------



3	Aurora - HPE Cray EX - Intel Exascale Compute Blade, Xeon CPU Max 9470 52C 2.4GHz, Intel Data Center GPU Max, Slingshot-11, Intel DOE/SC/Argonne National Laboratory United States	9,264,128	1,012.00	1,980.01	38,698
---	--	-----------	----------	----------	--------



4	JUPITER Booster - BullSequana XH3000, GH Superchip 72C 3GHz, NVIDIA GH200 Superchip, Quad-Rail NVIDIA InfiniBand NDR200, RedHat Enterprise Linux, EVIDEN EuroHPC/FZJ Germany	4,801,344	793.40	930.00	13,088
---	--	-----------	--------	--------	--------



5	Eagle - Microsoft NDv5, Xeon Platinum 8480C 48C 2GHz, NVIDIA H100, NVIDIA Infiniband NDR, Microsoft Azure Microsoft Azure United States	2,073,600	561.20	846.84	
---	--	-----------	--------	--------	--

www.top500.org

(June 2025)



6

HPC6 - HPE Cray EX235a, AMD Optimized 3rd Generation
EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, RHEL
8.9, HPE
Eni S.p.A.
Italy

3,143,520

477.90

606.97

8,461

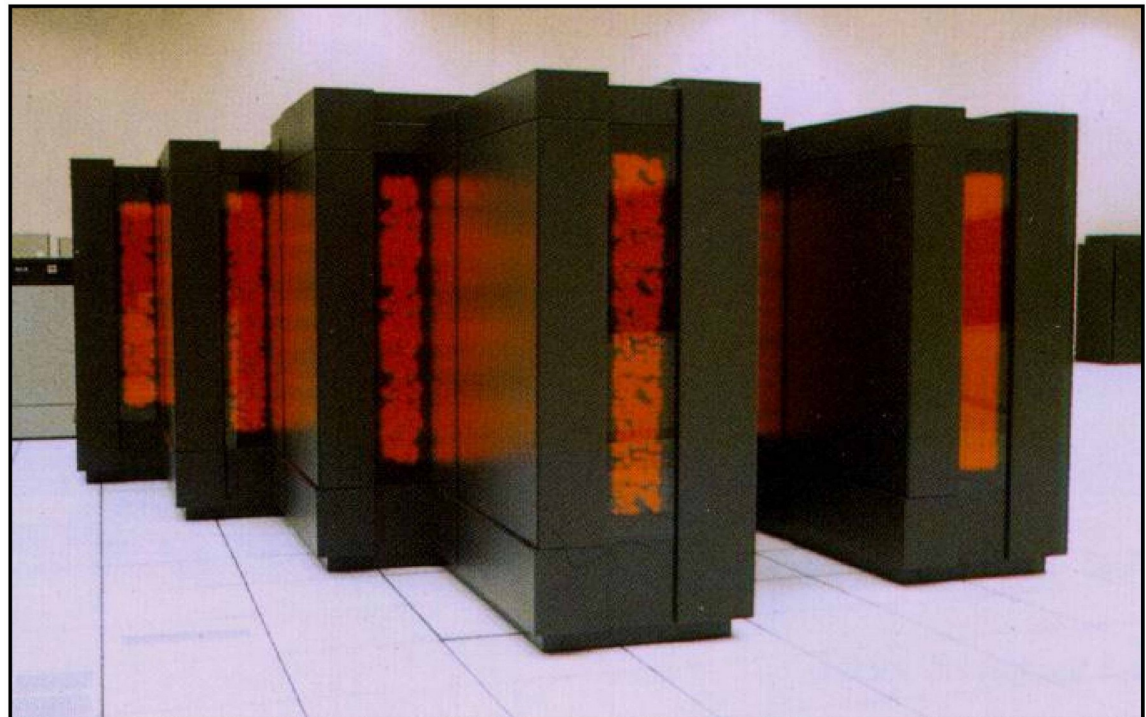


Cray 1 (year 1975)

Connection Machine
CM-2 (year 1987)



Connection Machine
CM-5 (year 1992)





IBM BlueGene/P (year 2007)



Atos BullSequana XH2000 “Leonardo”
(year 2022)

Parallel Architectures

The Computer Programme (BBC), “It's happening now”, 11/1/1982



https://archive.org/details/the_computer_programme_ep01

SANDIA ASCI RED

Date:

1996

Peak performance:

1.8Teraflops

Floor space:

150m²

Power consumption:

800.000 Watt



SANDIA ASCI RED

Date:

1996

Peak performance:

1.8Teraflops

Floor space:

150m²

Power consumption:

800.000 Watt

Sony

PLAYSTATION 3

Date:

2006

Peak performance:

>1.8Teraflops

Floor space:

0.08m²

Power consumption:

<200 Watt



Rules of thumb

- When writing parallel applications (especially on distributed-memory architectures) keep in mind that:
 - Computation is **fast**
 - Communication is **slow**
 - Input/output is **incredibly slow**

Recap

Shared memory

- Advantages:
 - Easier to program
 - Useful for applications with irregular data access patterns (e.g., graph algorithms)
- Disadvantages:
 - The programmer must take care of race conditions
 - Limited memory bandwidth

Distributed memory

- Advantages:
 - Highly scalable, provide very high computational power by adding more nodes
 - Useful for applications with strong locality of reference, high computation / communication ratio
- Disadvantages:
 - The programmer must take care of deadlocks
 - Latency of interconnect network
 - Difficult to program